

# TinyJ Assignment 3\*

The focus of this assignment is to complete the TJasn program so it will correctly *execute* the TinyJ virtual machine (VM) instructions that it generates. This assignment assumes you know what each TinyJ VM instruction should do when it is executed—if not, refer back to the “Effects of Executing Each TinyJ Virtual Machine Instruction” pages of “Asn-2-Memory-allocation-VM-instruction-set-and-hints.pdf” from Assignment 2.

## 1 How to Setup TinyJ Assignment 3

It is assumed you have successfully installed Assignment 2. You should complete the setup on *mars* and **test all the assignments on mars**. To setup Assignment 3, do the following step in your working directory for all platforms:

```
cp TJsolclasses/TJasn/virtualMachine/*instr.class TJasn/virtualMachine
```

The command above copies over Prof. Kong’s DEMO instruction classes into your TJasn/virtualMachine. After this step, you should be able to *execute* TinyJ programs. As an example:

```
java -cp . TJasn.TJ CS316ex9.java 9.out
```

You will see:

Using the following DEMO instruction classes:

```
PUSHSTATADDR PUSHNUM SAVETOADDR INITSTKFRM LOADFROMADDR ...
```

The result of execution should be the same as the output you get when you compile CS316ex9.java directly using:

```
javac CS316ex9.java [generate the byte codes]
```

```
java CS316ex9 [run the virtual machine to execute the byte codes]
```

## 2 How to Do TinyJ Assignment 3

For each VM instruction, there will be a corresponding file *\*instr.java* in your TJasn/virtualMachine directory. Every such file has an *execute()* method. In *STOPinstr.java*, *ADDTOPTRinstr.java*, *HEAPALLOCinstr.java*, *READINTinstr.java* and *NOPorDISCARDVALUEinstr.java*, *execute()* is written for you. In the other 29 *\*instr.java* files, the body of the *execute()* method contains a comment of the form */\* ???????? \*/ which you need to replace with code that executes the corresponding VM instruction*. Your code must make appropriate changes to the following data structures and must produce correct output (if any):

- Expression evaluation stack: `CodeInterpreter.EXPRSTACK[]`
- Data memory: `TJ.data[]`
- VM registers: `CodeInterpreter.ESP`, `.PC`, `.FP`, `.ASP`, etc..

As an example of how to complete an *execute()* method, consider the *execute()* method of *ADDinstr.java*. As Java (unlike C++) is guaranteed to evaluate expressions from left to right, the */\* ???????? \*/* in *ADDinstr.java* can be replaced with the following statement:

```
EXPRSTACK[--ESP-1] += EXPRSTACK[ESP];
```

After making this change, compile the program with:

```
javac -cp . TJasn/virtualMachine/ADDinstr.java
```

*ADDinstr.class* is generated and *replaces the DEMO version* of *ADDinstr.class* you copied into TJasn/virtualMachine.

To test the new version, try:

```
java -cp . TJasn.TJ CS316ex9.java 9.out
```

When you do this you should find that **ADD** is no longer on the list of **DEMO** instruction classes reported by *TJasn.TJ*! As before, when you are asked “Want debugging stop or post-execution dump? (y/n)” you should enter **n**.

---

\*Courtesy of Prof. Tatyung Kong

It is recommended that you work on the other instruction classes in the following order:

1. MUL, SUB, DIV, MOD, and CHANGESIGN, which are used by, e.g., CS316ex0.java.
2. NOT, AND, OR, EQ, NE, LT, GT, LE, GE, PUSHNUM, WITELNOP, WRITEINT, WRITESTRING, JUMP, JUMPONFALSE, PUSHSTATADDR, LOADFROMADDR, and SAVETOADDR, which are used by, e.g., CS316ex3.java.
3. PUSHLOCADDR, PASSPARAM, CALLSTATMETHOD, INITSTKFRM, and RETURN, all of which are used by, e.g., CS316ex9.java.

**Strong advice:** Each time you complete the `execute()` method of a file `xxxxinstr.java`, you should immediately compile the code and test your `execute()` method by using `TJasn.TJ` to execute a `CS316exk.java` program whose virtual machine code uses the instruction in question. If `TJasn.TJ` fails to execute that `CS316exk` program correctly, then your new `execute()` method is bugged and you need to fix it. Backup your correct `xxxxinstr.java` regularly.

### 3 How to Test Your Solution

When you have completed all the incomplete `execute()` methods and recompiled each of the modified `*instr.java` files, test your program on different source files (which should at least include the 16 `CS316exk.java` files) as follows:

```
java -cp . TJasn.TJ TinyJ-source-file-name output-file-name
Example: java -cp . TJasn.TJ CS316ex12.java 12.out
```

When it asks you “Want debugging stop or post-execution dump? (y/n)” you should enter **n**.

**Note:** If any DEMO instruction class is listed, the corresponding `*instr.java` file needs to be compiled! The runtime behavior should always be the same as when the same source file is compiled using `javac` and the resulting `.class` file is executed by a Java virtual machine. Also check that, for each `CS316exk.java` source file, a post-execution dump produced after execution of 1000 instructions is the same for your program as for Prof. Kong’s solution (see Assignment 2).

### 4 How to Submit

To submit:

1. Leave your final solution on *mars* in your `TJasn` directory, before midnight on the due date.
2. Make a backup copy for Assignment 3: `cp -r TinyJ TinyJ-assign3-bak`
3. Remove any incomplete `*instr.java` files you have **NOT** successfully completed from the `TJasn/virtualMachine` directory. **DO NOT REMOVE** `*instr.java` you have successfully completed.
4. Restore `*instr.class` to the DEMO version:  
`cp TJsolclasses/TJasn/virtualMachine/*instr.class TJasn/virtualMachine`
5. Recompile `*instr.java` you have completed to replace DEMO class files:  
`javac -cp . TJasn/virtualMachine/*instr.java.`
6. Any failed compilation will result in no credit for this assignment.
7. Students are allowed to work with other two students, but only one submission per group on Brightspace. Students have to stay in the same group for the entire project (3 assignments).
8. Each submission contains the absolute directory of your working directory, name(s) of student(s), **the list of your working VM instructions, and the list of exercise programs that can be properly executed.** If all works, just put ALL.

### 5 More about Implementation of TinyJ Virtual Machine

#### 5.1 CodeInterpreter.java

VM registers defined (see lines 14 – 18):

- PC contains the code memory address of the **next** instruction to be fetched for execution. (When that instruction is fetched, it is put into IR for execution.)

- ESP stores a count of the number of items that are currently on the expression evaluation stack—if `ESP > 0`, then `EXPRSTACK[ESP-1]` is the topmost element of the stack.
- FP is a pointer to the data memory location at offset 0 in the currently executing method activation’s stackframe. (Thus `FP+k` will be a pointer to the location at offset `k` in that stackframe.)
- ASP is a pointer to the *first unused* location in the stack-allocated part of data memory—i.e., ASP points to the first location above the currently allocated locations in that part of data memory. ASP must therefore be increased/decreased by  $k$  when  $k$  stackframe locations are allocated/deallocated.

The VM’s fetch-execute loop is in the `interpretCode( )` method of `CodeInterpreter.java` (line 48); this is called by `start( )` (line 247). Its most important lines are:

Line 75: `IR = TJ.generatedCode.get(oldPC = PC++);` and

Line 77: `IR.execute();`

Line 75 fetches the instruction stored (in code memory) at the address in PC and increments PC<sup>1</sup>. Line 77 *executes* the instruction that has just been fetched.

## 5.2 TJ.java

In `TJ.java`, the `ArrayList generatedCode` (line 30) and the array `data[]` (line 26) represent code and data memory, respectively. `TJ.generatedCode.get(a)` returns the instruction stored in code memory at address `a`. `TJ.data[a]` represents the data memory location whose address is `a`.

---

<sup>1</sup>The address of the instruction that is fetched is stored into `oldPC`—so when that instruction is being executed, and immediately afterwards, `oldPC` will contain the address of the instruction. When a debugging stop occurs—see the “Asn-3-how-to-debug.pdf” section below—the VM uses the address in `oldPC` to determine which instruction has just been executed. `PC - 1` will usually be equal to `oldPC`, but will sometimes be different because PC may be changed by execution of `JUMP`, `JUMPONFALSE`, `CALLSTATMETHOD`, or `RETURN`.